

PW EARNERS
DSA in C++

DPP — 8 (SOLUTIONS)

Topic: Binary Search

Multiple Choice Questions (Predict the Output)

Q1. Worst-case Time Complexity of Nested Logs

Correct Option: **C** ($O(\log^2 N)$)

Explanation: The outer loop doubles i at each step, running $O(\log N)$ times. Inside it, the inner loop divides j by 2 at each step, also running $O(\log N)$ times. Since they are nested, the total complexity is $O(\log N \times \log N) = O(\log^2 N)$.

.....

Q2. Lower and Upper Bound Index Difference

Correct Option: **B** (3)

Explanation: `lower_bound` for 4 points to its first occurrence at index 2. `upper_bound` for 4 points to the first element strictly greater than 4, which is 6 at index 5. Subtracting the iterators gives $5 - 2 = 3$.

.....

Q3. Custom Binary Search Logic

Correct Option: **C** (Infinite Loop (TLE))

Explanation: When $l = 0$ and $r = 1$, $mid = 0 + (1 - 0)/2 = 0$. Since `arr[0]` is 2, the condition `arr[mid] < 5` is true, setting $l = mid = 0$. The search space never shrinks, leading to an infinite loop.

.....

Q4. Dereferencing Lower Bound

Correct Option: **C** (40)

Explanation: `lower_bound` looks for the first element greater than or equal to 35. It finds 40 in the array. Dereferencing the returned pointer outputs 40.

.....

Q5. Crazy Question: Binary Search on Descending Array

Correct Option: **C** (Undefined Behavior / Unpredictable)

Explanation: `lower_bound` relies on a binary search algorithm that fundamentally requires the input range to be sorted in ascending order (or requires a custom comparator). Because the array here is in descending order, the behavior is strictly undefined.

.....

Q6. Find Smallest Letter Greater Than Target

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
```

```

4
5 char nextGreatestLetter(vector<char>& letters, char target) {
6     int l = 0, r = letters.size() - 1;
7     char res = letters[0]; // Wrap around default
8     while (l <= r) {
9         int mid = l + (r - 1) / 2;
10        if (letters[mid] > target) {
11            res = letters[mid];
12            r = mid - 1;
13        } else {
14            l = mid + 1;
15        }
16    }
17    return res;
18 }
19
20 int main() {
21     vector<char> letters = {'c', 'f', 'j'};
22     cout << nextGreatestLetter(letters, 'a') << endl;
23 }

```

Explanation : We perform a standard binary search. If `letters[mid]` is greater than the target, we record it and search left. Otherwise, we search right. Default initialization handles the wrap-around logic.

Q7. Minimum in Rotated Sorted Array

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int findMin(vector<int>& arr) {
6     int l = 0, r = arr.size() - 1;
7     while (l < r) {
8         int mid = l + (r - 1) / 2;
9         if (arr[mid] > arr[r]) {
10            l = mid + 1; // Minimum is in the right unsorted portion
11        } else {
12            r = mid; // Minimum is mid or in the left sorted portion
13        }
14    }
15    return arr[l];
16 }
17
18 int main() {
19     vector<int> arr = {3, 4, 5, 1, 2};
20     cout << findMin(arr) << endl;
21 }

```

Explanation : By comparing `arr[mid]` with `arr[r]`, we can deduce which half is unsorted and thus contains the "cliff" or the minimum element.

Q8. Rotation Count

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int findRotationCount(vector<int>& arr) {
6      int l = 0, r = arr.size() - 1;
7      while (l < r) {
8          int mid = l + (r - l) / 2;
9          if (arr[mid] > arr[r]) {
10             l = mid + 1;
11         } else {
12             r = mid;
13         }
14     }
15     return l; // The index of the minimum element represents
               // rotation count
16 }
17
18 int main() {
19     vector<int> arr = {15, 18, 2, 3, 6, 12};
20     cout << findRotationCount(arr) << endl;
21 }

```

Explanation : The number of times a sorted array is rotated right directly equals the index of its minimum element.

.....

Q9. Frequency of Target Element

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int countOccurrences(vector<int>& arr, int target) {
7      auto lb = lower_bound(arr.begin(), arr.end(), target);
8      auto ub = upper_bound(arr.begin(), arr.end(), target);
9      return distance(lb, ub);
10 }
11
12 int main() {
13     vector<int> arr = {1, 1, 2, 2, 2, 2, 3};
14     cout << countOccurrences(arr, 2) << endl;
15 }

```

Explanation : Utilizing STL `lower_bound` and `upper_bound` finds the span of the target element in $O(\log N)$ time.

.....

Q10. Check Majority Element

```

1  #include <iostream>
2  #include <vector>

```

```

3 #include <algorithm>
4 using namespace std;
5
6 bool isMajority(vector<int>& arr, int x) {
7     int n = arr.size();
8     auto it = lower_bound(arr.begin(), arr.end(), x);
9     if (it == arr.end() || *it != x) return false;
10
11     int first_idx = distance(arr.begin(), it);
12     // Check if the element at first_idx + n/2 is also x
13     if (first_idx + n / 2 < n && arr[first_idx + n / 2] == x) {
14         return true;
15     }
16     return false;
17 }
18
19 int main() {
20     vector<int> arr = {1, 2, 3, 3, 3, 3, 10};
21     cout << (isMajority(arr, 3) ? "true" : "false") << endl;
22 }

```

Explanation : We find the first occurrence of X . If X is a majority element (appears $> N/2$ times), then the element exactly $N/2$ steps ahead of its first occurrence must also be X .

Q11. First 1 in an Infinite Binary Array

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int firstOneInfinite(vector<int>& arr) {
6     int l = 0, r = 1;
7
8     // Expand bounds exponentially until a '1' is included
9     while (arr[r] == 0) {
10         l = r;
11         r = r * 2;
12     }
13
14     int ans = -1;
15     // Standard Binary Search in the valid range
16     while (l <= r) {
17         int mid = l + (r - l) / 2;
18         if (arr[mid] == 1) {
19             ans = mid;
20             r = mid - 1; // Keep searching left for earlier
                           occurrence
21         } else {
22             l = mid + 1;
23         }
24     }
25     return ans;
26 }

```

```

27
28 int main() {
29     vector<int> arr(1000000, 0);
30     arr[4] = 1; arr[5] = 1; // Simulated Infinite Array structure
31     cout << firstOneInfinite(arr) << endl;
32 }

```

Detailed Explanation: Because the array is infinite, we cannot use `arr.size()` to determine our right boundary (r). To overcome this in logarithmic time, we define a sliding window starting at sizes 1 ($l = 0, r = 1$) and double the bounds ($r = r \times 2$) continuously until we capture a '1' within our scope (`arr[r] == 1`). Once we have definitively boxed the transition point between '0's and '1's within a finite $[l, r]$ range, we apply standard Binary Search to pinpoint the exact first occurrence of '1'.

Q12. Missing Number in Arithmetic Progression

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int findMissing(vector<int>& arr) {
6     int n = arr.size();
7     int diff = (arr[n - 1] - arr[0]) / n; // Formula to find the
      expected jump
8     int l = 0, r = n - 1;
9
10    while (l <= r) {
11        int mid = l + (r - l) / 2;
12        // Check if missing is strictly to the right
13        if (arr[mid] == arr[0] + mid * diff) {
14            l = mid + 1;
15        } else {
16            r = mid - 1;
17        }
18    }
19    // 'l' lands right where the pattern broke
20    return arr[0] + l * diff;
21 }
22
23 int main() {
24     vector<int> arr = {2, 4, 8, 10, 12, 14};
25     cout << findMissing(arr) << endl;
26 }

```

Explanation : If no elements were missing, `arr[mid]` would equal `arr[0] + mid * diff`. If this holds true, the left side of the array is perfectly intact, meaning the missing number lies strictly in the right half.

Q13. Floor and Ceil in Sorted Array

```

1 #include <iostream>
2 #include <vector>

```

```

3 using namespace std;
4
5 void floorAndCeil(vector<int>& arr, int x) {
6     int l = 0, r = arr.size() - 1;
7     int floor_val = -1, ceil_val = -1;
8
9     while (l <= r) {
10         int mid = l + (r - 1) / 2;
11         if (arr[mid] == x) {
12             cout << "Floor = " << arr[mid] << ", Ceil = " << arr[mid]
13                 << endl;
14             return;
15         } else if (arr[mid] < x) {
16             floor_val = arr[mid]; // Potential Floor
17             l = mid + 1;
18         } else {
19             ceil_val = arr[mid]; // Potential Ceil
20             r = mid - 1;
21         }
22     }
23     cout << "Floor = " << floor_val << ", Ceil = " << ceil_val <<
24         endl;
25 }
26
27 int main() {
28     vector<int> arr = {1, 2, 8, 10, 10, 12, 19};
29     floorAndCeil(arr, 5);
30 }

```

Explanation : We maintain two candidate variables. Whenever `arr[mid] < x`, it becomes a valid floor candidate and we push right. When `arr[mid] > x`, it becomes a valid ceil candidate and we push left.

Q14. Search in a Bitonic Array

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int findPeak(vector<int>& arr) {
6     int l = 0, r = arr.size() - 1;
7     while (l < r) {
8         int mid = l + (r - 1) / 2;
9         if (arr[mid] > arr[mid + 1]) r = mid;
10        else l = mid + 1;
11    }
12    return l;
13 }
14
15 int binarySearch(vector<int>& arr, int l, int r, int target, bool
16     isAscending) {
17     while (l <= r) {
18         int mid = l + (r - 1) / 2;

```

```

18         if (arr[mid] == target) return mid;
19
20         if (isAscending) {
21             if (arr[mid] < target) l = mid + 1;
22             else r = mid - 1;
23         } else {
24             if (arr[mid] > target) l = mid + 1;
25             else r = mid - 1;
26         }
27     }
28     return -1;
29 }
30
31 int main() {
32     vector<int> arr = {1, 3, 8, 12, 4, 2};
33     int target = 4;
34
35     int peak = findPeak(arr);
36     int idx = binarySearch(arr, 0, peak, target, true);
37     if (idx == -1) {
38         idx = binarySearch(arr, peak + 1, arr.size() - 1, target,
39                             false);
40     }
41
42     cout << idx << endl;
43 }

```

Detailed Explanation: A Bitonic Array strictly increases up to a singular "peak" element and then strictly decreases. We split the problem into two distinct phases to maintain $O(\log N)$ complexity. First, we locate the index of the peak element using Binary Search (comparing mid to $mid+1$). This peak acts as a partition. Second, we execute an ascending Binary Search on the left sub-array $[0, \text{peak}]$. If the target isn't found there, we execute a specialized descending Binary Search on the right sub-array $[\text{peak} + 1, N - 1]$.

Q15. K-th Element of Two Sorted Arrays

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int kthElement(vector<int>& arr1, vector<int>& arr2, int n, int m,
7                int k) {
8      // We heavily ensure that Binary Search happens on the smaller
9      // array
10     if (n > m) return kthElement(arr2, arr1, m, n, k);
11
12     int low = max(0, k - m), high = min(k, n);
13
14     while (low <= high) {
15         int cut1 = low + (high - low) / 2;
16         int cut2 = k - cut1;

```

```

16     int l1 = (cut1 == 0) ? -1e9 : arr1[cut1 - 1];
17     int l2 = (cut2 == 0) ? -1e9 : arr2[cut2 - 1];
18     int r1 = (cut1 == n) ? 1e9 : arr1[cut1];
19     int r2 = (cut2 == m) ? 1e9 : arr2[cut2];
20
21     // Proper partition found
22     if (l1 <= r2 && l2 <= r1) {
23         return max(l1, l2);
24     }
25     else if (l1 > r2) {
26         high = cut1 - 1; // Left block of arr1 is too big,
27                          // shrink it
28     }
29     else {
30         low = cut1 + 1; // Need more elements from arr1
31     }
32     return -1;
33 }
34
35 int main() {
36     vector<int> arr1 = {2, 3, 6, 7, 9};
37     vector<int> arr2 = {1, 4, 8, 10};
38     int k = 5;
39     cout << kthElement(arr1, arr2, arr1.size(), arr2.size(), k) <<
40         endl;
41 }

```

Detailed Explanation: This problem is a variation of finding the median of two sorted arrays. We want to virtually build a "left half" containing exactly K elements across both arrays such that every element in this left half is smaller than everything in the virtual "right half". To achieve $O(\log(\min(M, N)))$, we strictly binary search only on the smaller array (let's say `arr1`). If we take `cut1` elements from `arr1`, we are mathematically forced to take `cut2 = k - cut1` elements from `arr2`. We compare the edge boundary variables (`l1`, `l2` against `r1`, `r2`). A valid crossover (`l1 <= r2` and `l2 <= r1`) means our combination is perfect, and the K^{th} element is simply the largest of the left-side items: `max(l1, l2)`.

All the best! Keep Coding ♡